



Production Readiness

What's left for the website & apps to serve real users

Reviewed July 2026
Customer · Driver · Merchant apps
+ Website & backend

Where things stand

The product has crossed from **polished demo** into a **working system with a real backend**. A Node.js/Express service persists everything to SQLite (schema is Postgres-compatible), issues real JWT sessions, and is the source of truth the MQTT bus reflects. Auth, payments, payouts, push, dispatch, ticket scanning and WhatsApp support are all **code-complete** — most run on **dev stubs** awaiting real provider credentials and hardening. The remaining work is **flip the stubs to real providers, harden the infrastructure, and ship to the stores** — integration, not rewrites.

✓ DONE SINCE THE LAST REVIEW

- ✓ **Real backend + database** — Express REST + SQLite persistence for orders, menus, bookings, tickets, reviews, favorites, notifications, payments, payouts, users & sessions. Survives restarts.
- ✓ **Real accounts & auth** — phone OTP + JWT access/refresh sessions, plus a Twilio Verify WhatsApp/SMS flow. Roles wired across all three apps.
- ✓ **Payments + payouts** — full charge / refund / payout API + ledger tables. Auto-completes in dev; ready for a real PSP.
- ✓ **Push notifications** — expo-notifications + push service and token registry across all three apps.
- ✓ **Dispatch / matching engine** — presence-aware timed job offers, auto-match, reassignment, no-driver handling.
- ✓ **Ticket redemption** — merchant scan-and-validate screen. **Live menus** — merchant edits publish live to customers. **WhatsApp support desk** — webhook + agent dashboard + 24h rule.

TIER 1 Hard blockers — can't take real money or real users without these

- 1 **Provider credentials & go-live switch.** Code is built but running on dev stubs that auto-complete. Needs a real **payment provider** (Paynow / EcoCash / OneMoney / Zimswitch / cards), a real **SMS/WhatsApp sender** (Twilio), and `NODE_ENV=production` so stubs stop short-circuiting.
- 2 **Production broker + infrastructure + secrets.** Still the dev Aedes broker with open auth and `localhost / 192.168.x.x` hardcoded. Needs a hosted **MQTT broker with TLS (wss://)** + ACLs, env-based config, a strong `JWT_SECRET`, and a **managed Postgres** (schema already compatible).
- 3 **App-store distribution.** Apps run in Expo Go. Needs **EAS builds**, Apple + Google developer accounts, store listings & review (background-location justification), real icons/splash, and OTA updates.
- 4 **Payments hardening.** Beyond wiring the PSP: capture/refund reconciliation against the provider, webhook signature verification, an auditable ledger, and idempotency on charge/refund.

TIER 2 Core product completeness

- 5 **Real background GPS.** Driver publishes real GPS, but background tracking needs a standalone build with the iOS background-location entitlement + an Android foreground service. (Simulated-GPS is demo-only.)
- 6 **Merchant self-onboarding.** Live menus exist, but there's no self-serve signup → verify → menu/inventory/pricing/hours flow to add a merchant without hand-seeding.

7 Real geocoding + ETAs. Addresses use hardcoded coords and the rate-limited public OSRM demo server. Needs geocoding + a production routing provider, plus a Google Maps API key for Android.

8 Order edge cases. Cancellation, partial refunds, failed-payment retries, and out-of-stock mid-order are only partially handled end-to-end.

TIER 3 Trust, ops & legal

- 9 **Admin / ops console.** No surface yet to manage users, resolve disputes, issue refunds, suspend accounts, or monitor live orders.
- 10 **Legal & compliance.** Terms, privacy policy, data-protection stance, real driver background checks (currently mock), insurance, and age/ID checks for event & alcohol sales.
- 11 **Security hardening.** Rate limiting, input validation on every endpoint, webhook signature verification (done for WhatsApp; needed for payments), secrets management, and automated DB backups.
- 12 **Monitoring & delivery.** Error tracking (Sentry), uptime/alerting, product analytics, log aggregation, and CI/CD.

TIER 4 Polish

- 13 **Resilience & reach.** Offline cache for flaky connectivity, accessibility (screen readers, contrast, touch targets), internationalisation, test coverage beyond the bus-contract smoke test, and cold-start/performance passes.

Website — launch checklist

Marketing site + Business/Driver dashboards (`website/`)

The site shares the same backend. Before launch it specifically needs: **HTTPS + a real domain**, its dashboards wired to the **authenticated API** (not mock data), **SEO / meta + Open Graph**, **cookie / privacy + legal pages**, **analytics**, and a **CDN** for assets.

Recommended go-live sequence

1. **Managed Postgres + hosted broker (TLS/ACLs) + env-based config & secrets** — the foundation everything assumes.
2. **Wire the real payment provider + reconciliation/webhooks** — the revenue path and biggest risk.
3. **Real SMS/WhatsApp sender for OTP + support** — gates sign-up.
4. **EAS store builds + background-GPS entitlements** — get installable apps out.
5. **Security hardening + monitoring/backups** — before real traffic.
6. **Admin console + legal/compliance** — required to operate and resolve disputes.
7. **Merchant self-onboarding, geocoding/routing, edge cases** — scale the catalogue and ops.
8. **Polish** — offline, accessibility, i18n, tests.

The `SpotlyClient` SDK and the Postgres-compatible schema were shaped so each item slots in behind existing interfaces — these are integration and hardening tasks, not rewrites.